

Apple Unified Log Predicates in iLEAPP: The Reference

Every Unified Log message predicate iLEAPP searches for, artifact by artifact, with the processes involved, the iOS versions each pattern was observed on, the sources that documented them, and the caveats that keep you honest.

Alexis Brignoni
Updated August 14, 2026

Live version: leapps.org/blog-post?post=2026-08-01-unified-log-predicate-reference

This is the companion to the [Unified Logs workflow guide](#). That article covers how to acquire, preserve, and process the logs. This one answers the question that comes right after processing: what exactly is iLEAPP looking for, and why should you trust any of it?

[Download the printable PDF edition.](#)

As of iLEAPP's main branch on August 14, 2026, the Unified Logs module registers **36 artifacts** driven by **235 unique message predicates** and four log categories. Every single one of them meets a standard I want to be transparent about, because nobody should take "the tool found it" as an answer:

1. **Documented:** the pattern comes from published research that was fetched and verified, not from memory or folklore. The source is cited on the artifact, in the code, and here.
2. **Observed:** the pattern was matched against real full file system extractions. Four images did the heavy lifting: an iPhone 8 Plus on iOS 16.5 (19.4 million log records), an iPhone 11 Pro on iOS 17.1 (30.4 million records), an iPhone 12 on iOS 18.7 (20.5 million records), and an iPhone on iOS 26.5.2 (15.1 million records).

Most patterns meet both bars. Where a pattern is documented but the event never occurred in our images, the artifact notes say **documented-only**, and so does this guide. An empty artifact is not evidence of absence, and a populated one still deserves validation on your device and iOS version before it goes in a report.

Three caveats apply to everything below, so I will say them once instead of two hundred and thirty five times:

- **Payloads redact.** On production devices the dynamic values in many messages render as <private>. The predicates anchor on the static message text on purpose. When a value does survive in the clear, treasure it.
- **Strings drift.** Apple changes log messages between iOS versions. We already carry version-specific variants for several patterns, and iOS 26 changed others. A miss on your image may just mean a new spelling nobody has documented yet.
- **Retention is short and uneven.** Most entries live hours to days, not the mythical 30. Tim Korver and Lionel Notari have both published on this. Acquire early.

The verbatim, always-current predicate list lives in the [iLEAPP source module](#). The lists below were generated from that file, not retyped, which is the only honest way to publish two hundred plus strings. The one exception is the dialed numbers artifact, whose clauses pair a category with a message pattern and so are written out here by hand.

How the module is put together

Two artifacts do the heavy lifting, and everything else filters their output:

- **logarchive** imports every record into the LAVA database. On the iPhone 12 image that is 20.5 million rows. LAVA-only, for obvious reasons.
- **logarchive artifacts** runs every predicate in a single pass over that table and materializes the matches. Also LAVA-only.

The remaining 33 artifacts each carve their slice out of that filtered set. Thirty produce standard reports; the three highest-volume ones (keyboard activity, biometric sensor events, touchscreen events) are LAVA-only because a two-hundred-thousand-row HTML file helps nobody.

Now the reference, grouped by what the evidence speaks to.

Human presence and handling

[logarchive biometric sensor events](#)

Sensor-level entries from the Face ID and Touch ID stacks: camera frames with face-detected, attention, and glasses flags, face-to-device distance readings, SpringBoard face-in-view notices, and finger-on/finger-off events on home button devices. Sensor entries record what the hardware saw, not an unlock decision; pair them with the lock artifacts. This is the evidence a person cannot anticipate or stage, which is exactly why Tim Korver ranks it so high on evidential strength.

Observed: Face ID families on iOS 17.1 and 18.7 (11,307 PearlCam frames on the iPhone 12 alone); Touch ID finger events on the iOS 16.5 iPhone 8 Plus. Documented-only: the home button press form (iOS 12 era).

Sources: Thesis Friday [#1](#), [#12](#), [#10](#), [#22](#).

Predicates:

- %PearlCamFrameReceived%
- %getFaceDetectInfo%
- %[User Presence Monitor]%
- %kAppleBiometricFinger%
- %Home Button Was Pressed%

logarchive touchscreen events

Fingers physically on glass: digitizer contact presence per finger, per-app touch statistics windows, touch attention events, and tap-to-wake. The "was the phone being handled at 14:32" artifact.

Observed: iOS 16.5, 17.1, 18.7.

Sources: [Thesis Friday #14](#), [Lionel Notari on touch events](#).

Predicates:

- %contact _ presence:%
- %touchstats%
- %received tapToWake%
- %AttentionAwareness.Touch%

logarchive pocket state

The front infrared sensor logging when the device is face-down or stowed, and when it comes back out. Directly speaks to "the phone was in my pocket."

Observed: iOS 17.1 and 18.7. The iOS 16.5 iPhone 8 Plus produced none, which fits its older sensor package.

Sources: [Ian Whiffin's Doppler research](#), [Notari's SQL queries post](#).

Predicates:

- %Doppler in pocket state%
- %PocketState changed%

logarchive lock status

The original lock artifact: screen lock and unlock, device lock status, and biometric match messages.

Sources: community research consolidated in the module; see the workflow guide.

Predicates:

- %Screen did lock%
- %ScreenOn changed%
- %Screen shut off%
- %screen is locked%
- %screen is unlocked%
- %Device unlocked%
- %Device lock status%
- %Biometric match complete%

logarchive unlock sessions and method

The newer, sharper companion: session durations ("Was unlocked for N seconds"), authentication requests with type and outcome (type 1 passcode, type 2 biometric, and failures log too), chronod locked-state transitions, keybag and APFS volume unlock, and locks from the side button.

Observed: iOS 16.5, 17.1, 18.7.

Sources: [Notari's unlock post](#), [Thesis Friday #12](#).

Predicates:

- %Screen did unlock (Was locked for%
- %Screen did lock (Was unlocked for%
- %Processed authentication request%
- %Transition: locked ->%
- %apfs is being UN-locked%
- %lock button source%

Communication and input

logarchive call events

Telephony from the OS side, independent of the call history database: callservicesd call tracking start and end, Phone app open requests that name the originating process (a touch, Siri, or a Bluetooth head unit, which matters enormously in distracted-driving cases), Phone tab navigation, and keypad tone requests where actionIDs 1200 through 1209 map to keypad digits 0 through 9. Yes, that means a hand-dialed number can sometimes be reconstructed from sound requests. The number payloads in these particular entries redact to <private>, which is not the end of the story: see the dialed numbers artifact below.

Observed: iOS 16.5, 17.1, 18.7. The tone requests come from mediaserverd in the research and audiomxd on iOS 18.7, and only when keypad sounds are enabled.

Sources: Notari on [making a call](#) and [watchOS calls](#).

Predicates:

- %Started tracking call%
- %Dialed call%
- %Call started outgoing%

- %All calls ended%
- %Received trusted open application request%
- %Resuming to tab type%
- %tab bar tab changed%
- %Incoming Request : actionID 120%

logarchive dialed numbers

The number itself, in plain text, and the typing that produced it. Two separate families, and they are worth keeping straight.

The first is CommCenter's `call.provider` transaction log. A call setup block carries `kActionType: 0` and a `kPhoneNumber` field holding the dialed number unredacted; a teardown block carries `kActionType: 2` and the same `kUuid`, which is what pairs the two. The artifact also collects the `Call(StatusUpdate)` state chain, because the research uses that chain to separate a call that connected from one that only dialed. The second is MobilePhone's `ContactSearchManager`, which logs `Searching for <digits>` and `Search cancelled for <digits>` on each change to the dial field, so the sequence reconstructs the digits going in one at a time.

Tim Korver's finding underneath all of this is worth stating on its own, because it changes how you read a log: `<private>` **is a property of one logging call, not of the underlying data**. The same transaction arrives from `callservicesd` redacted and is written out in full by CommCenter microseconds later. Stopping at `<private>` is stopping too early.

Observed, and this is the interesting part: each family showed up on exactly one of four images swept for both.

Image	iOS	Log records	call.provider	kActionType blocks	ContactSearchManager
iPhone 8 Plus	16.5	19,419,414	1,534	0	0
iPhone 11 Pro	17.1	30,362,747	172	0	0
iPhone 12	18.7	20,491,691	1,482	0	752
iPhone	26.5.2	15,146,256	328	8	0

On the iOS 26.5.2 image, three of those eight blocks are `kActionType: 0` and each carries a phone number in the clear in E.164 form, each followed by a `kActionType: 2` block sharing its `kUuid`. That reproduces the published structure on a different device, a different iOS version, and a different carrier region, which is the kind of corroboration I like. Of the remaining five blocks, three are those `kActionType: 2` teardowns and two carry values 1 and 7, which no source I read defines, so the artifact reports them as stored rather than guessing. On the iOS 18.7 image, 301 of the 752 `ContactSearchManager` entries are the `Searching for` and `Search cancelled for` pairs, and the digit strings lengthen one step at a time up to a ten-digit value, exactly as described.

Say what that table does and does not show. Every image had telephony activity, so `call.provider` is not the variable. But four images is four images: no absence in that table shows a family is unavailable on that release, and I am not claiming a version boundary from it.

Two caveats from the source that I will not soften. A setup block with no matching teardown is a dialed attempt; it does not establish that a call connected. And `ContactSearchManager` fires for digits entered on the device keypad but not for a number typed on a CarPlay screen, so its presence places entry on the handset while its absence only rules the handset keypad out. Contacts, Recents, and Siri were not tested.

Two implementation notes, since this reference exists to show the workings. The whole `call.provider` category is collected rather than a message pattern, because the teardown block carries nothing but `kActionType` and `kUuid` and there is no distinctive text to anchor on; the volume stays small, as the table shows. Its sibling category `call`

comes along for the same reason and is where the `Call(StatusUpdate)` chain actually lives, which cost me a run to find out: 199 records on the iOS 26.5.2 image, and worth reading for the surrounding call bookkeeping. And every clause is scoped to a category rather than left as a bare substring, because bare substrings over twenty million records find things you did not mean. `%kPhoneNumber%` on its own also matches locationd's `kPhoneNumberStatusNotification` under category Emergency, which carries no number at all: 7 of those on the iOS 16.5 image and 44 on the iOS 17.1 one, against 3 genuine setup blocks in the whole sweep. `%Searching for%` on its own matched 808 unrelated records on the iOS 26.5.2 image, not one of them a dial field. Scoping to the category drops both to zero false positives.

Source: [Thesis Friday #24](#).

Predicates:

- category `call.provider` with `%kPhoneNumber%`
- category `call.provider` with `%kActionType%`
- category `call` with `%Call(StatusUpdate)%`
- category `ContactSearchManager`

logarchive keyboard activity

Per-keystroke evidence of active typing, attributed to the app receiving it: keyboard touch signposts (documented under subsystem `UIKitCore`, logging under `com.apple.TextInput` on iOS 18.7) plus keyboard sound requests for character, delete, and modifier keys. Text content is not recorded; the typing activity is. Over 200,000 signpost rows on a single image, so this one is LAVA-only.

Observed: iOS 16.5, 17.1, 18.7.

Sources: [Thesis Friday #17](#), [Notari on typing in WhatsApp](#).

Predicates:

- category `KeyboardSignposts`
- `%Incoming Request : actionID 1104%`
- `%Incoming Request : actionID 1155%`
- `%Incoming Request : actionID 1156%`

logarchive dictation

Voice input instead of typed input: dictation start with the keyboard language code, begin and end feedback, and the assistantd record type that distinguishes keyboard dictation from a Siri request.

Observed: iOS 18.7. Zero on the other two images, which simply means nobody dictated.

Source: [Notari on dictation](#).

Predicates:

- `%DictationConnection startDictation%`
- `%Dictation did begin%`
- `%Dictation did end%`
- `%CSAudioRecordTypeDictation%`

logarchive notification interactions

Notifications the user did something with: removals, group expansion, cell default actions (a tap-through into the app), long-look presentation, and replies from the notification itself. Content is not recorded; interaction is.

Observed: removal entries on all three images. Documented-only: tap-through, expansion, and reply forms.

Source: [Notari's SQL queries post](#).

Predicates:

- %removing notification request%
- %expanding notification group%
- %notification cell executing default action%
- %will present long look%
- %action reply for notification%

Application activity

logarchive executed apps

The original launch artifact: icon taps, application launches, and workspace transitions.

Predicates:

- %Allowing tap for icon view%
- %Launching application%
- %transition source:%

logarchive app focus and lifecycle

The finer-grained companion: which app held focus moment to moment (an empty value means the home screen), cold starts versus resumes ("Bootstrapping ... with intent foreground-interactive" means the app was fully closed; iOS 16 spells it `application<`, iOS 17 and later spell it `app<`), scene lifecycle changes, icon taps tied to launches, and force-kills from the app switcher. A force-kill right after an event of interest is itself worth noticing.

Observed: iOS 16.5, 17.1, 18.7.

Sources: Notari's [unlock](#) and [SQL queries](#) posts.

Predicates:

- %/device/app/inFocus%
- %Bootstrapping app<%
- %Bootstrapping application<%
- %killed from app switcher%
- %elementWithFocusBundleID changed%
- %Icon tapped%
- %Initiating launch from icon view%
- %Scene lifecycle state did change%

logarchive interface navigation

Deliberate handling between app launches: Control Center, Today view, widget visibility, home screen page scrolling.

Observed: iOS 16.5, 17.1, 18.7.

Source: [Notari's SQL queries post](#).

Predicates:

- %Control Center launched%
- %Control Center Visible%
- %Setting visibility of widget%
- %Today view overlay%
- %user-initiated scroll%

Connectivity

logarchive wifi status

The broadest artifact in the module, and it earned it: power state, association, reachability, SSIDs, forgotten and removed networks, scans, joins, known-network activity, per-network MAC randomisation records you can line up against router logs, keychain password retrieval, auto-join entries that carry the SSID in the clear, link loss, and per-network session duration. The 2026-08-01 additions distinguish a network picked by hand from an auto-join, which is intent evidence.

Observed: original set plus WFMacRandomisation on iOS 18.7; password retrieval, auto-join with SSID, link loss, and connection time on iOS 16.5 and 17.1. Documented-only: manual association.

Sources: Notari's [WiFi and airplane mode](#) and [SQL queries](#) posts.

Predicates:

- %WiFi state changed:%
- %Toggled WiFi state%
- %is WiFi associated?%
- %link status changed%
- %reachability changed%
- %ISNetworkObserver%
- %ForgetSSID%
- %en0: SSID%
- %Removing Lease SSID%
- %SysMon: WiFi state changed:%
- %WiFiManagerClientRemoveNetworkWithReason:%
- %WiFiSecurityRemovePassword%
- %AlwaysOnWifi:%
- %WiFiDeviceManagerSetNetworks:%
- %Scanning For Broadcast found:%
- %Scanning Remaining Channels%

- %WiFiSettlementObserver _handleScanResults%
- %Attempting to join%
- %WiFiLQAMgrSetCurrentNetwork: Joined SSID:%
- %Preparing background scan request for %
- %WiFiNetworkPrepareKnownBssList%
- %to list of known networks%
- %{AUTOJOIN, SCAN*} Scanning 2Ghz Channels found:%
- %{AUTOJOIN, SCAN*} Scanning 5Ghz Channels found:%
- %WFMacRandomisation%
- %manual association%
- %Copy password for Network%
- %Attempting auto join association%
- %Link went down%
- %Total connection time%

logarchive bluetooth status

Bluetooth power and state, device connections and disconnections, hands-free activity, call state updates, A2DP streaming, and link quality.

Predicates:

- %Bluetooth state changed%
- %Sending new bluetooth state%
- %Bluetooth state changed PoweredOn%
- %ServiceManager disconnection result for%
- %Device type is%
- %is asking to connect device%
- %Received connection result for%
- %Received disconnection result for%
- %Received handsfree disconnection%
- %Sending ring notification for call%
- %Accepting incoming audio connection%
- %Received voice audio connected%
- %Stopping A2DP audio streaming%
- %Bluetooth A2DP device%
- %Bluetooth Daemon: A2DP streaming%
- %Starting Media connection to device%
- %Received voice disconnection%
- %Disconnecting audio from device%
- %Audio was already disconnected%
- %Toggled Bluetooth state from%

- %CUBluetoothDevice%
- %handsfree device disconnected%
- %handsfree device connected%
- %Bluetooth state updated%
- %Bluetooth power is now off%
- %Bluetooth state%
- %Sending call state update%
- %A2DP LinkQualityReport%

logarchive bluetooth pairing

Discovery and pairing rather than reconnection: bluetoothd Device found: CBDevice records that carry the accessory name, Bluetooth address, and product identifiers in the clear (the iOS 16.5 image logged a neighbor's AirPods by name, which tells you exactly how much this sees), plus pairing session lifecycle entries including rapportd Continuity pairing.

Observed: discovery and "Pairing completed" on iOS 16.5 and 17.1. Documented-only: the pairing dialog forms (pairing started, numeric comparison, SDP), because no new pairing happened in our windows.

Source: [Notari on Bluetooth pairing](#).

Predicates:

- %Device found: CBDevice%
- %pairing complete%
- %pairing started%
- %numeric comparison%
- %Running SDP%

logarchive personal hotspot

Tethering and wireless-modem state changes.

Predicates:

- %Tethering is now enabled with%
- %Received notification that wireless modem state changed%
- %Previous tethering state was%

logarchive SIM and cellular state

SIM slot status values, cellular data network type changes, and itunestored's view of the network type, which catches WiFi-to-cellular fallback. Slot status records state at logging time, not the moment a card moved.

Observed: iOS 16.5, 17.1, 18.7.

Source: [Notari's WiFi and airplane mode post](#).

Predicates:

- %kCTSIMSupportSIMStatus%
- %dataNetwork changed to%

- %disabling dataNetwork%
- %ISNetworkObserver: Set network type%

logarchive airplane mode

All the enabled/disabled/active/inactive variants, plus the 2026-08-01 toggle forms where the logging process tells you how it was done: SpringBoard means Control Center, Preferences means Settings, assistant means Siri. The iOS 26 CoreTelephony state reads are deliberately not collected; they are polls, not events, and the artifact notes say so.

Sources: [Notari](#), [Thesis Friday #13](#).

Predicates:

- %Airplane Mode is now 1%
- %Airplane Mode is now On%
- %Setting airplane mode to true%
- %Airplane mode now active%
- %enabling airplanemode%
- %Airplane mode changed%
- %Airplane Mode is now 0%
- %Airplane Mode is now Off%
- %Setting airplane mode to false%
- %Airplane mode now inactive%
- %Airplane mode Disabled%
- %Toggle AirPlane Mode state%
- %Setting airplane mode enabled%

logarchive AirDrop

The device's rotating AirDrop ID (observed in the clear: Current AirDrop ID is f6f8ae502871), discoverability scanning mode (Everyone, Contacts Only, Off), SharingDaemon state dumps with device model and battery level, share sheet activation, and transfer entries. AirDrop IDs rotate, so an ID ties activity together only within a session.

Observed: ID, scanning mode, and state dumps on iOS 17.1; share sheet and startSending on iOS 18.7.

Documented-only: incoming transfer and accept/decline forms.

Source: [Sarah Edwards](#), [Quarantine Edition Entry 11](#).

Predicates:

- %AirDrop ID%
- %SharingDaemon State%
- %Scanning mode%
- %startSending%
- %New incoming transfer%
- %alertLog: idx:%
- %Activating com.apple.sharing.sharesheet%

Device state and power

logarchive power events

Boot and orderly shutdown markers: the kernel iBoot version line at startup, SpringBoard's orientation-deferral shutdown notice, and locationd shutting down. Pair these with the Sysdiagnose shutdown.log artifacts, which as of this week also carry Kaspersky's iShutdown triage heuristics: per-process shutdown delays and location indicators for the directories their Pegasus, Reign, and Predator analysis found malware delaying reboots from.

Observed: iOS 16.5 and 17.1; locationd form on 18.7.

Sources: [Notari's unlock post](#), [Kaspersky's shutdown.log research](#).

Predicates:

- %iBoot version%
- %Deferring device orientation updates for reason: shutdown%
- %locationd shutting down%

logarchive time change

Clock shifts, the system's significant time change broadcast, and the timed manual-time-setting entries that record a clock set by hand on the device. That last family is anti-forensics evidence and it is documented-only so far; none of our images had a hand-set clock, and honestly I am glad.

Sources: [Notari on clock trust](#).

Predicates:

- %Time change: Clock shifted by%
- %Significant time change%
- %TMSetManualTime%
- %setting manual time%

logarchive battery state

Charge level changes posted by powerd and battery info updates from PowerUIAgent. A charge curve corroborates charging claims and powered-on state.

Observed: iOS 16.5, 17.1, 18.7.

Source: [Notari's SQL queries post](#).

Predicates:

- %Battery capacity change posted%
- %battery info changed to%

logarchive USB and power connections

Cable attach and detach at two layers: powerexperienced plugin state changes and the kernel cable-detect shim, including the VBUS power and CON_DET physical-connection states. Present : 0 is the detach signal. Fair warning from someone who processes his own evidence: your acquisition produces these too.

A note on why the VBUS pattern is here, since I got this one wrong first. The CarPlay research quotes that line without the shim prefix, so I expected we were missing it. We were not: on our images every VBUS line carried the IOAccessoryUSBConnectShim prefix the artifact already matched, 30 records on iOS 18.7 and 40 on iOS 17.1, none without it. The pattern is version insurance for a release that drops the prefix, not a gap that was closed. The artifact notes say the same thing.

Observed: iOS 16.5, 17.1, 18.7.

Sources: Thesis Friday [#9](#) and [#20](#).

Predicates:

- %plugin state changed to%
- %IOAccessoryUSBConnectShim%
- %USB Power (VBUS) Present%

Media, audio, and camera

logarchive media playback

The MediaRemote category: now-playing state with the originating app's bundle id, playback state changes, and route information. Sarah Edwards documented duration, elapsed time, and AirPlay target names in these entries. Activity and alibi evidence.

Observed: iOS 16.5, 17.1, 18.7.

Source: [Sarah Edwards, Quarantine Edition Entry 9](#).

Predicates:

- category MediaRemote

logarchive audio status

Playback state, volume-button presses, volume changes, and playback queue invalidation.

Predicates:

- %AudioQueueIsPlaying%
- %VolumeIncrement%
- %rawVolumeIncreasePress%
- %rawVolumeDecreasePress%
- %Volume active%
- %PlaybackQueueInvalidation%
- %volumeValueDidChange%

logarchive audio routes

Where the sound went: receiver, speaker, or a Bluetooth device, for calls and other audio sessions. The research shows Bluetooth routes carrying the accessory MAC address, which is hands-free-versus-handset evidence during a call.

Observed: iOS 16.5, 17.1, 18.7.

Source: [Notari on calls and audio output](#).

Predicates:

- %vaemConfigurePVMSettings%
- %vaemVADRouteChangeListener%
- %cmsmActivateEndpointFromRouteDescription%
- %currently activating endpoint%

logarchive camera capture

The capture chain from shutter to library: capture mode changes, moment capture begin and commit, still image capture, and assetsd adding the result to the photo library, with IMG_ filenames when not redacted. Ties a specific photo's creation to hands-on camera use, and survives the photo's later deletion for as long as the log entry lives.

Observed: the full chain on iOS 18.7, elements on 16.5 and 17.1.

Source: [Notari's SQL queries post](#).

Predicates:

- %will change to: Photo%
- %MomentCapture%
- %Still image capture type%
- %IrisWillBeginCapture%
- %added photo to library%
- %added video to library%
- %Created asset IMG%

logarchive flashlight

Flashlight controller and AVFlashlight activity, including the spaced and unspaced AVFoundation variants because iOS could not pick one.

Predicates:

- %[Flashlight Controller]%
- %<<<<AVFlashlight>>>>-%
- %<<<< AVFlashlight >>>>%

Motion and vehicle

logarchive motion state transitions

Motion-state transition messages.

Predicates:

- %Motion State Transition:%

logarchive driving state

Vehicular motion classification: wifid CMMotionActivity driving start and stop, locationd vehicular episode markers, Driving Focus engagement, and the pedestrian-after-driving alarm that marks the moment someone exits a vehicle. Two boundaries the research draws and I will repeat: these entries do not distinguish driver from passenger, and the classifier fires on other transport too. It also came with a lesson in humility: the original pattern for the locationd marker was missing a space and never matched real data. Fixed now, validated now.

Observed: iOS 16.5 and 17.1.

Source: [Notari on driving](#).

Predicates:

- %MotionState: Driving%
- %vehicularStartTime%
- %PedestrianAfterDriving%
- %Engaging Driving%
- %com.apple.donotdisturb.mode.driving%
- %ATXModeDrivingFeaturizer%

logarchive navigation

Apple Maps guidance prompts: route start, maneuvers, distance callouts, arrival.

Predicates:

- %Starting route to%
- %Proceed to the%
- %Proceed to\%
- %Turn right%
- %Turn left%
- %roundabout%
- %first exit%
- %Stay in the%
- %parking lot for%
- %of a mile%
- %In about%
- %then arrive%
- %your destination%
- %At the light%
- %Arrived\%

logarchive CarPlay session

The connection handshake: the airplayd DirectLink notice that marks a wired session rather than a wireless one, CarKit session authentication and activation state, CarPlayApp vehicle identifier entries, and the wifid record that carries the vehicle's reported model and manufacturer in the clear.

This one is documented-only, and I want to be blunt about it. Every pattern comes from Tim Korver's handshake research, revised for iOS 26.6 on an iPhone 14. None of it has been seen in our images, because none of them contain a CarPlay session. I swept the full marker set across complete iOS 17.1 and 18.7 extractions and got zero for every pattern, and an earlier sweep of an iOS 16.5 extraction found nothing either. The `com.apple.carkit` and `CarPlayApp` matches that do appear are subsystem and process mentions in unrelated entries, the same trap as the thousands of `CARSession` hits that turned out to be `thermalmonitord`'s `carSessionActive` flag. So treat anything this artifact returns as unconfirmed until you have seen it on a device you know used CarPlay, and if you do, please tell me.

The source's caveats are worth repeating: the vehicle identifier is assigned by the device rather than read from the car, so it needs the surrounding session to attribute it to a vehicle; the `CarKit` line carries its meaning in the `isAuthenticated` and `isActive` values rather than the message name, and fires around a thousand times per session; the `FrontBoard` bootstrap marker appeared in only one run of three, so its absence shows nothing; and the research covered one vehicle over wired USB, with first-time pairing and wireless sessions untested. The `Stark` subsystem the whole feature was built on no longer exists as of iOS 26.6.

Source: [Thesis Friday #20](#).

Predicates:

- `%Found USB DirectLink%`
- `%session isAuthenticated%`
- `%vehicle ID%`
- `%Persisting widget state%`
- `%WiFiDeviceManagerSetCarPlaySessionState%`
- `%CarPlay session vehicle inform%`
- `%CarPlay Connection Event%`

Emergency

logarchive emergency SOS engine

`sosd` status broadcasts and flow state, including the paired-device trigger entry documented from an Apple Watch. Read the boundary carefully: these broadcasts appeared on devices with no known SOS use, so presence alone does not show an SOS call. Payloads redact.

Observed: broadcasts and flow entries on iOS 16.5 and 17.1. Documented-only: the paired-device trigger.

Source: [Thesis Friday #19](#).

Predicates:

- `%broadcasting SOSStatus%`
- `%flowStartedOnEitherDevice%`
- `%sosTriggeredOnPairedDevice%`

The wide net

logarchive artifacts is the collection everything above filters from, and it holds predicates that do not have a dedicated report yet: screenshots, charger connection state, CarPlay connection events, device orientation, Siri

speech request sessions, Apple Account authentication, contact-detail presence, walking bouts, accessory connections, brightness, ringer state, and the Emergency SOS claw gesture. They are all in LAVA under the broad collection. When one of them earns a dedicated artifact, it will move up into this list.

What did not make the cut, and why

Transparency section, because nobody tells you what their tool is missing:

- **CarPlay per-vehicle attribution is in, but unproven.** The artifact above now collects the handshake, including the vehicle identifier and the model and manufacturer record, straight from the published research. Nothing in our corpus exercises it. Thousands of promising "CARSession" matches turned out to be thermalmonitord's `carSessionActive` flag.
- **TCC permission changes.** On iOS those entries are volatile and never reach the logarchive. TCC.db is the durable source. This one is macOS-only evidence.
- **evaluatePowerMode and the CoreTelephony airplane reads.** Present by the tens of thousands, but they are state polls, not events. Collecting them would bury signal in noise.
- **The dialed-number families each sit on one image.** `kPhoneNumber` on iOS 26.5.2, `ContactSearchManager` on iOS 18.7, neither on the other. I would like an image where both appear together, and images that place the boundary of each. If you have a device you can dial from, this is a twenty-minute test.
- **Still hunting validation data for:** AirDrop transfer accept and decline, the Bluetooth pairing dialog sequence, watch-relayed call invites, manual clock setting, Back Tap gestures, and a dialed number entered from Contacts, Recents, or Siri rather than the keypad. All documented, all in the module as documented-only patterns or noted in the research, all waiting for an image where the event actually happened.

That last list is an invitation. If you have a test device and twenty minutes, generate one of those events, pull the logs, and check the patterns. If they hold, send a pull request or just tell me. If they drifted, definitely tell me. This module got its last two waves of artifacts exactly that way: published research plus somebody bothering to validate it against real extractions. The LEAPPs are free and open source, yesterday, today, tomorrow, and forever, and they grow when the community feeds them.

Good stuff lives in these logs. Go find yours.

For questions, updates, and my current contact links, visit abrignoni.github.io.